

Laboratorio 2

Pregunta 1

En el contexto de Cheapest, el endpoint GET evaluado combina lectura histórica, promociones y disponibilidad por zona, mientras el POST confirma pedidos en eventos de alta demanda. Si solo pudiera optimizar uno antes de un pico comercial, ¿cuál priorizaría y por qué?

POST (/logistics/pedidos / ASR 2).

Priorizaría el endpoint POST (/logistics/pedidos) por su impacto crítico y directo en los ingresos del negocio: un fallo en la escritura significa una venta perdida, mientras que la lentitud del GET solo retrasa temporalmente la vista del catálogo. Técnicamente, el POST presenta el mayor riesgo de incumplimiento del ASR 2 durante el pico de promociones (5000 req/min), ya que sus escrituras transaccionales saturarán rápidamente el pool de 10 conexiones por defecto de la base de datos; por el contrario, el GET es de lectura pura y PostgreSQL lo paraleliza mejor. Finalmente, mientras que optimizar el GET requiere cambios rápidos y de bajo costo como agregar índices, asegurar el POST exige mejoras arquitectónicas complejas y costosas (como implementación de colas o rediseño transaccional), haciéndolo la inversión prioritaria indispensable antes de enfrentar un evento de alta demanda.

Pregunta 2

Diseñe una distribución de datos que haga realista el escenario de Cheapest en hora pico (tiendas con comportamientos heterogéneos, zonas con distinta densidad de pedidos y promociones activas). Tome como punto de partida el warm-up

Para que el escenario de hora pico sea realista, la distribución de datos no puede ser uniforme: en la vida real hay tiendas y zonas de alta rotación que concentran la mayoría de los pedidos y promociones (ciudades grandes como Bogotá o CDMX), mientras el resto compra poco. Por eso en el load-seed.yaml usamos una distribución Pareto donde el 20% de tiendas/zonas "cabeza" concentra el 80% de la actividad.

Sesgo de una distribución uniforme: si todas las tiendas y productos tienen la misma probabilidad de ser consultados, la base de datos nunca enfrenta el desgaste real de que muchas peticiones caigan sobre las mismas filas al mismo tiempo. Esto hace que el punto de inflexión medido salga más alto y optimista de lo que realmente sería en producción.

Estrategia A (GET / ASR1): concentrar las consultas en la tienda y zona "cabeza" (80% del tráfico de prueba). Valida si el problema está en la falta de índices y en un plan de consulta que se repite sobre las mismas filas calientes.

Estrategia B (POST / ASR2): dispersar ampliamente tiendas y productos entre los pedidos simultáneos. Valida si el cuello de botella es el número limitado de conexiones a la base de datos, sin importar qué fila se escriba.

ENTIDAD	CANTIDAD TOTAL
TIENDAS	200
ZONAS	10
CATÁLOGOS POR ZONA	30 (300 catálogos / 10 zonas)
PRODUCTOS	1.200
PROMOCIONES ACTIVAS	150
PEDIDOS HISTÓRICOS POR TIENDA	100 (20.000 pedidos / 200 tiendas, en promedio — no distribuidos igual entre todas)

Pregunta 3

La matriz propuesta aumenta la carga de forma escalonada, pero no necesariamente separa bien las causas de degradación. ¿Qué cambiaría en el diseño experimental para distinguir si el quiebre proviene principalmente de saturación del pool de conexiones, consultas SQL ineficientes, límites del generador de carga? Defina un diseño alternativo con variables controladas y los resultados esperados para cada hipótesis.

La matriz actual sube la carga de forma escalonada, pero eso no dice *por qué* se rompe el sistema. Para distinguir la causa real, proponemos tres experimentos que cambian una sola variable a la vez, dejando todo lo demás igual:

1. Pool de conexiones: repetir el escenario "Estrés fuerte" subiendo solo el tamaño del pool (10 → 30 → 100). Si el error% del POST baja notablemente, confirma que el pool era la causa; si no cambia, se descarta.

2. Índices faltantes: correr la misma carga de "Operación normal" del GET sin y con los 4 índices que faltan (pedidos.tiendald, items_pedido.productold, catalogos.zona, disponibilidad_zona.productold). Si el p99 baja claramente con los índices, esa era la causa.
3. Límite del generador de carga: correr la misma carga nominal (300-450 threads) una vez con JMeter y otra con el script Python. Si los resultados difieren para la misma carga, el límite viene del generador y no del backend — se confirma revisando CPU/memoria de la máquina de pruebas.

Lo clave del diseño es ese aislamiento: una variable por experimento, todo lo demás constante, y siempre las mismas métricas (p99, error%) para poder comparar resultados entre sí.

Pregunta 4

Para los escenarios de más de 450 threads debe generar un script de carga con ayuda de un agente de IA. En un equipo real, varias personas generarían código con IA sobre el mismo repositorio de Cheapest y es fácil que el estilo, las convenciones y las decisiones de arquitectura se vuelvan inconsistentes. Investigue qué técnicas existen para lograr que los agentes de IA generen código siguiendo de forma consistente los patrones y reglas de desarrollo definidas por todo el equipo.

- Implementar archivos de configuración persistentes a nivel de repositorio (como .github/copilot-instructions.md para Copilot, .cursorrules para Cursor o CLAUDE.md). La IA lee estos archivos automáticamente en cada sesión, asimilando al instante las reglas de arquitectura, convenciones de nombres y manejo de errores específicas del proyecto.
- Establecer la revisión de Pull Requests (PR) como un paso humano obligatorio, respaldado por un pipeline de CI con pruebas automatizadas. Esta es la red de seguridad definitiva para atrapar inconsistencias o alucinaciones independientes del agente utilizado.
- Documentar, versionar y compartir los prompts dentro del equipo (tal como se exige en los entregables de este laboratorio). Esto estandariza las peticiones y asegura que todos los miembros partan del mismo contexto al interactuar con la IA.

Ejecución de GET

Escenario	Endpoint	Corrida	Threads / users	Ramp-up	Duración	p99 (ms)	p95 (ms)	Throughput	Error %
Smoke test	GET	1	5	5s	—	49	49	1.2/sec	0.00
Baja carga	GET	1	30	10s	—	85	49	3.1/sec	0.00
Carga media	GET	1	100	20s	—	54	49	5.0/sec	0.00
Operación normal	GET	1	450	50s	—	21	20	9.0/sec	0.00
Operación normal	GET	2	450	50s	—	21	19	9.0/sec	0.00
Operación normal	GET	3	450	50s	—	22	19	9.0/sec	0.00
Operación normal	GET	4	450	50s	—	20	21	9.0/sec	0.00
Operación normal	GET	5	450	50s	—	21	19	9.0/sec	0.00
Alta carga	GET	1	1500	75s	60s	45363	28189	106.09 /sec	0.00
Alta carga	GET	2	1500	75s	60s	54800	32380	92.39 /sec	0.00
Alta carga	GET	3	1500	75s	60s	47391	29687	109.46 /sec	0.00
Muy alta carga	GET	1	3000	100s	60s	64704	41497	87.36 /sec	0.00
Muy alta carga	GET	2	3000	100s	60s	65225	40931	97.33 /sec	0.00
Muy alta carga	GET	3	3000	100s	60s	68288	43867	90.51 /sec	0.00
Estrés	GET	1	7500	150s	60s	104529	66655	60.4 /sec	0.00
Estrés	GET	2	7500	150s	60s	103822	67510	61.03 /sec	0.00

<i>Estrés</i>	GET	3	7500	150s	60s	101746	66028	57.09 /sec	0.00
<i>Estrés fuerte (pico)</i>	GET	1	18000	200s	60s	174906	117042	76.40	0.00
<i>Estrés fuerte (pico)</i>	GET	2	18000	200s	60s	164365	109355	78.44 /sec	0.00
<i>Estrés fuerte (pico)</i>	GET	3	18000	200s	60s	166731	112589	73.63 /sec	0.00
<i>Estrés fuerte (pico)</i>	GET	4	18000	200s	60s	166353	114776	68.90 /sec	0.00
<i>Estrés fuerte (pico)</i>	GET	5	18000	200s	60s	163471	109719	74.37 /sec	0.00

Evidencia

Smoke test

Plan de Pruebas Grupo GET Request GET Request View Results Tree Summary Report Aggregate Report (Percentile) Graph Results Grupo POST Request																																																			
Aggregate Report Name: Aggregate Report (Percentiles de Latencia) Comments: Write results to file / Read from file Filename Browse... Log/Display Only: ☐ Errors ☐ Successes Configure <table> <tr> <th>Label</th><th># Samples</th><th>Average</th><th>Median</th><th>90% Line</th><th>95% Line</th><th>99% Line</th><th>Min</th><th>Maximum</th><th>Error %</th><th>Through...</th><th>Receive...</th><th>Sent KB/...</th></tr> <tr> <td>GET Req...</td><td>5</td><td>47</td><td>48</td><td>48</td><td>49</td><td>49</td><td>44</td><td>49</td><td>0.00%</td><td>1.2/sec</td><td>2.06</td><td>0.27</td></tr> <tr> <td>TOTAL</td><td>5</td><td>47</td><td>48</td><td>48</td><td>49</td><td>49</td><td>44</td><td>49</td><td>0.00%</td><td>1.2/sec</td><td>2.06</td><td>0.27</td></tr> </table>													Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Through...	Receive...	Sent KB/...	GET Req...	5	47	48	48	49	49	44	49	0.00%	1.2/sec	2.06	0.27	TOTAL	5	47	48	48	49	49	44	49	0.00%	1.2/sec	2.06	0.27
Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Through...	Receive...	Sent KB/...																																							
GET Req...	5	47	48	48	49	49	44	49	0.00%	1.2/sec	2.06	0.27																																							
TOTAL	5	47	48	48	49	49	44	49	0.00%	1.2/sec	2.06	0.27																																							

Baja carga

Plan de Pruebas Grupo GET Request GET Request View Results Tree Summary Report Aggregate Report (Percentiles de Latencia) Graph Results Grupo POST Request																																																			
Aggregate Report Name: Aggregate Report (Percentiles de Latencia) Comments: Write results to file / Read from file Filename Browse... Log/Display Only: ☐ Errors ☐ Successes Configure <table> <tr> <th>Label</th><th># Sampl...</th><th>Average</th><th>Median</th><th>90% Line</th><th>95% Line</th><th>99% Line</th><th>Min</th><th>Maximu...</th><th>Error %</th><th>Throug...</th><th>Receive...</th><th>Sent KB...</th></tr> <tr> <td>GET Re...</td><td>30</td><td>44</td><td>43</td><td>49</td><td>49</td><td>85</td><td>38</td><td>85</td><td>0.00%</td><td>3.1/sec</td><td>5.14</td><td>0.67</td></tr> <tr> <td>TOTAL</td><td>30</td><td>44</td><td>43</td><td>49</td><td>49</td><td>85</td><td>38</td><td>85</td><td>0.00%</td><td>3.1/sec</td><td>5.14</td><td>0.67</td></tr> </table>													Label	# Sampl...	Average	Median	90% Line	95% Line	99% Line	Min	Maximu...	Error %	Throug...	Receive...	Sent KB...	GET Re...	30	44	43	49	49	85	38	85	0.00%	3.1/sec	5.14	0.67	TOTAL	30	44	43	49	49	85	38	85	0.00%	3.1/sec	5.14	0.67
Label	# Sampl...	Average	Median	90% Line	95% Line	99% Line	Min	Maximu...	Error %	Throug...	Receive...	Sent KB...																																							
GET Re...	30	44	43	49	49	85	38	85	0.00%	3.1/sec	5.14	0.67																																							
TOTAL	30	44	43	49	49	85	38	85	0.00%	3.1/sec	5.14	0.67																																							

Carga media

Aggregate Report												
Name: Aggregate Report (Percentiles de Latencia)												
Comments:												
Write results to file / Read from file												
Filename			Browse...		Log/Display Only:		☐ Errors	☐ Successes	Configure			
Label	# Sampl...	Average	Median	90% Line	95% Line	99% Line	Min	Maximu...	Error %	Throug...	Receive...	Sent KB...
GET Re...	100	43	43	47	49	54	30	90	0.00%	5.0/sec	8.38	1.10
TOTAL	100	43	43	47	49	54	30	90	0.00%	5.0/sec	8.38	1.10

Operación Norma – Primera Ejecución

Aggregate Report												
Name: Aggregate Report (Percentiles de Latencia)												
Comments:												
Write results to file / Read from file												
Filename			Browse...		Log/Display Only:		☐ Errors	☐ Successes	Configure			
Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Through...	Receive...	Sent KB/...
GET Req...	450	17	17	19	20	21	13	62	0.00%	9.0/sec	8.67	1.97
TOTAL	450	17	17	19	20	21	13	62	0.00%	9.0/sec	8.67	1.97

Operación Normal – Segunda Ejecución

Aggregate Report												
Name: Aggregate Report (Percentiles de Latencia)												
Comments:												
Write results to file / Read from file												
Filename			Browse...		Log/Display Only:		☐ Errors	☐ Successes	Configure			
Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Through...	Receive...	Sent KB/...
GET Req...	450	16	17	19	19	21	12	41	0.00%	9.0/sec	8.67	1.97
TOTAL	450	16	17	19	19	21	12	41	0.00%	9.0/sec	8.67	1.97

Operación Norma – Tercera Ejecución

Aggregate Report												
Name: Aggregate Report (Percentiles de Latencia)												
Comments:												
Write results to file / Read from file												
Filename			Browse...		Log/Display Only:		☐ Errors	☐ Successes	Configure			
Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Through...	Receive...	Sent KB/...
GET Req...	450	16	16	19	19	22	12	48	0.00%	9.0/sec	8.67	1.97
TOTAL	450	16	16	19	19	22	12	48	0.00%	9.0/sec	8.67	1.97

Operación Norma – Cuarta Ejecución

Aggregate Report

Name: Aggregate Report (Percentiles de Latencia)

Comments:

Write results to file / Read from file

Filename: Browse... Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Through...	Receive...	Sent KB/...
GET Req...	450	16	17	19	20	21	13	40	0.00%	9.0/sec	8.67	1.97
TOTAL	450	16	17	19	20	21	13	40	0.00%	9.0/sec	8.67	1.97

Operación Norma – Quinta Ejecución

Aggregate Report

Name: Aggregate Report (Percentiles de Latencia)

Comments:

Write results to file / Read from file

Filename: Browse... Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Through...	Receive...	Sent KB/...
GET Req...	450	16	17	19	19	21	11	41	0.00%	9.0/sec	8.67	1.97
TOTAL	450	16	17	19	19	21	11	41	0.00%	9.0/sec	8.67	1.97

Alta Carga – Primera Ejecución

```
st-api-load_tests> python scripts/load_test.py --endpoint GET --users 1500 --ramp-up 75 --duration 60
=====
Resumen de la prueba – GET
=====
Total requests:      15718
Duracion real (s):   148.2
Throughput (req/s):  106.09
Latencia promedio:   7997.11 ms
Latencia p95:        28189.66 ms
Latencia p99:        45363.54 ms
Error %:              0.00%
Resultados guardados en: results_get.csv
=====
```

Alta Carga – Segunda Ejecución

```
st-api-load_tests> python scripts/load_test.py --endpoint GET --users 1500 --ramp-up 75 --duration 60
=====
Resumen de la prueba – GET
=====
Total requests:      13611
Duracion real (s):   147.3
Throughput (req/s):  92.39
Latencia promedio:   8705.35 ms
Latencia p95:        32380.80 ms
Latencia p99:        54800.77 ms
Error %:              0.00%
Resultados guardados en: results_get.csv
=====
```

Alta Carga – Tercera Ejecución

```

st-api-load_tests> python scripts/load_test.py --endpoint GET --users 1500 --ramp-up 75 --duration 60
=====
Resumen de la prueba – GET
=====
Total requests:      16219
Duracion real (s):   148.2
Throughput (req/s):  109.46
Latencia promedio:   7945.13 ms
Latencia p95:        29687.25 ms
Latencia p99:        47391.81 ms
Error %:             0.00%
Resultados guardados en: results_get.csv
=====

```

Muy Alta Carga – Primera Ejecución

```

st-api-load_tests> python scripts/load_test.py --endpoint GET --users 3000 --ramp-up 100 --duration 60
=====
Resumen de la prueba – GET
=====
Total requests:      18143
Duracion real (s):   207.7
Throughput (req/s):  87.36
Latencia promedio:   11783.41 ms
Latencia p95:        41497.90 ms
Latencia p99:        64704.00 ms
Error %:             0.00%
Resultados guardados en: results_get.csv
=====

```

Muy Alta Carga – Segundo Ejecución

```

st-api-load_tests> python scripts/load_test.py --endpoint GET --users 3000 --ramp-up 100 --duration 60
=====
Resumen de la prueba – GET
=====
Total requests:      19603
Duracion real (s):   201.4
Throughput (req/s):  97.33
Latencia promedio:   11477.66 ms
Latencia p95:        40931.65 ms
Latencia p99:        65225.56 ms
Error %:             0.00%
Resultados guardados en: results_get.csv
=====

```

Muy Alta Carga – Tercera Ejecución

```

st-api-load_tests> python scripts/load_test.py --endpoint GET --users 3000 --ramp-up 100 --duration 60
=====
Resumen de la prueba – GET
=====
Total requests:      18713
Duracion real (s):   206.7
Throughput (req/s):  90.51
Latencia promedio:   11857.40 ms
Latencia p95:        43867.54 ms
Latencia p99:        68288.55 ms
Error %:             0.00%
Resultados guardados en: results_get.csv
=====

```

Estrés – Primera Ejecución

```
st-api-load_tests> python scripts/load_test.py --endpoint GET --users 7500 --ramp-up 150 --duration 60
=====
Resumen de la prueba – GET
=====
Total requests:      22059
Duracion real (s):   367.4
Throughput (req/s):  60.04
Latencia promedio:   18863.25 ms
Latencia p95:        66655.14 ms
Latencia p99:        104529.08 ms
Error %:             0.00%
Resultados guardados en: results_get.csv
=====
```

Estrés – Segunda Ejecución

```
st-api-load_tests> python scripts/load_test.py --endpoint GET --users 7500 --ramp-up 150 --duration 60
=====
Resumen de la prueba – GET
=====
Total requests:      22233
Duracion real (s):   364.3
Throughput (req/s):  61.03
Latencia promedio:   18899.94 ms
Latencia p95:        67510.86 ms
Latencia p99:        103822.50 ms
Error %:             0.00%
Resultados guardados en: results_get.csv
=====
```

Estrés – Tercera Ejecución

```
st-api-load_tests> python scripts/load_test.py --endpoint GET --users 7500 --ramp-up 150 --duration 60
=====
Resumen de la prueba – GET
=====
Total requests:      21069
Duracion real (s):   369.1
Throughput (req/s):  57.09
Latencia promedio:   18942.02 ms
Latencia p95:        66028.06 ms
Latencia p99:        101746.16 ms
Error %:             0.00%
Resultados guardados en: results_get.csv
=====
```

Estrés Fuerte – Primera Ejecución

```
st-api-load_tests> python scripts/load_test.py --endpoint GET --users 18000 --ramp-up 200 --duration 60
=====
Resumen de la prueba – GET
=====
Total requests:      23584
Duracion real (s):   308.7
Throughput (req/s):  76.40
Latencia promedio:   33842.88 ms
Latencia p95:        117042.08 ms
Latencia p99:        174906.32 ms
Error %:             0.00%
Resultados guardados en: results_get.csv
=====
```

Estrés Fuerte – Segunda Ejecución

```

st-api-load_tests> python scripts/load_test.py --endpoint GET --users 18000 --ramp-up 200 --duration 60
=====
Resumen de la prueba – GET
=====
Total requests:      23985
Duracion real (s):   305.8
Throughput (req/s):  78.44
Latencia promedio:   32446.34 ms
Latencia p95:        109355.99 ms
Latencia p99:        164365.26 ms
Error %:             0.00%
Resultados guardados en: results_get.csv
=====

```

Estrés Fuerte – Tercera Ejecución

```

st-api-load_tests> python scripts/load_test.py --endpoint GET --users 18000 --ramp-up 200 --duration 60
=====
Resumen de la prueba – GET
=====
Total requests:      23304
Duracion real (s):   316.5
Throughput (req/s):  73.63
Latencia promedio:   33484.48 ms
Latencia p95:        112589.34 ms
Latencia p99:        166731.75 ms
Error %:             0.00%
Resultados guardados en: results_get.csv
=====

```

Estrés Fuerte – Cuarta Ejecución

```

st-api-load_tests> python scripts/load_test.py --endpoint GET --users 18000 --ramp-up 200 --duration 60
=====
Resumen de la prueba – GET
=====
Total requests:      21249
Duracion real (s):   308.4
Throughput (req/s):  68.90
Latencia promedio:   33944.12 ms
Latencia p95:        114776.66 ms
Latencia p99:        166353.42 ms
Error %:             0.00%
Resultados guardados en: results_get.csv
=====

```

Estrés Fuerte – Quinta Ejecución

```

st-api-load_tests> python scripts/load_test.py --endpoint GET --users 18000 --ramp-up 200 --duration 60
=====
Resumen de la prueba – GET
=====
Total requests:      22868
Duracion real (s):   307.5
Throughput (req/s):  74.37
Latencia promedio:   32937.04 ms
Latencia p95:        109719.20 ms
Latencia p99:        163471.54 ms
Error %:             0.00%
Resultados guardados en: results_get.csv
=====

```

Ejecución de POST

Escenario	Endpoint	Corrida	Threads / users	Ramp-up	Duración	p99 (ms)	p95 (ms)	Throughput	Error %
Smoke test	POST	1	5	5s	—	88	88	1.2/sec	0.00
Baja carga	POST	1	30	10s	—	86	42	3.2/sec	0.00
Carga media	POST	1	100	20s	—	39	34	5.0/sec	0.00
Operación normal	POST	1	450	50s	—	26	24	9.0/sec	0.00
Operación normal	POST	2	450	50s	—	25	24	9.0/sec	0.00
Operación normal	POST	3	450	50s	—	25	24	9.0/sec	0.00
Operación normal	POST	4	450	50s	—	25	24	9.0/sec	0.00
Operación normal	POST	5	450	50s	—	26	24	9.0/sec	0.00
Alta carga	POST	1	1500	75s	60s	58616	37321	75.13 /sec	0.02
Alta carga	POST	2	1500	75s	60s	24395	21669	69.77 /sec	0.14
Alta carga	POST	3	1500	75s	60s	50234	32295	89.34 /sec	0.01
Muy alta carga	POST	1	3000	100s	60s	75590	48153	66.94 /sec	0.01
Muy alta carga	POST	2	3000	100s	60s	68169	46160	61.00 /sec	0.00
Muy alta carga	POST	3	3000	100s	60s	40828	28251	54.15 /sec	0.05
Estrés	POST	1	7500	150s	60s	34569	30273	47.06 /sec	38.84

<i>Estrés</i>	POST	2	7500	150s	60s	110505	73381	43.84 /sec	0.01
<i>Estrés</i>	POST	3	7500	150s	60s	114520	75555	40.90 /sec	0.01
<i>Estrés fuerte (pico)</i>	POST	1	18000	200s	60s	67846	37608	80.84 /sec	52.32
<i>Estrés fuerte (pico)</i>	POST	2	18000	200s	60s	76624	36618	83.77 /sec	50.71
<i>Estrés fuerte (pico)</i>	POST	3	18000	200s	60s	48198	32146	91.00 /sec	55.11
<i>Estrés fuerte (pico)</i>	POST	4	18000	200s	60s	44118	31811	90.16 /sec	49.86
<i>Estrés fuerte (pico)</i>	POST	5	18000	200s	60s	51184	28547	102.41 /sec	69.59

Evidencia

Smoke Test

Plan de Pruebas

Grupo GET Request

Grupo POST Request

POST Request

HTTP Header Manager

View Results Tree

Summary Report

Aggregate Report (Percentile)

Graph Results

Aggregate Report

Name:Aggregate Report (Percentiles de Latencia)

Comments:

Write results to file / Read from file

Filename

Browse...

Log/Display Only:

Errors

Successes

Configure

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Through...	Receive...	Sent KB/...
POST Re...	5	50	41	50	88	88	35	88	0.00%	1.2/sec	2.55	0.00
TOTAL	5	50	41	50	88	88	35	88	0.00%	1.2/sec	2.55	0.00

Baja Carga

Aggregate Report

Name: Aggregate Report (Percentiles de Latencia)

Comments:

Write results to file / Read from file

Filename: Browse... Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Through...	Receive...	Sent KB/...
POST Re...	30	35	33	38	42	86	30	86	0.00%	3.1/sec	6.37	0.00
TOTAL	30	35	33	38	42	86	30	86	0.00%	3.1/sec	6.37	0.00

Carga Media

Aggregate Report

Name: Aggregate Report (Percentiles de Latencia)

Comments:

Write results to file / Read from file

Filename: Browse... Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Through...	Receive...	Sent KB/...
POST Re...	100	28	29	32	34	39	20	78	0.00%	5.0/sec	10.37	0.00
TOTAL	100	28	29	32	34	39	20	78	0.00%	5.0/sec	10.37	0.00

Operación Norma – Primera Ejecución

Aggregate Report

Name: Aggregate Report (Percentiles de Latencia)

Comments:

Write results to file / Read from file

Filename: Browse... Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Through...	Receive...	Sent KB/...
POST Re...	450	21	22	24	24	26	17	54	0.00%	9.0/sec	18.55	0.00
TOTAL	450	21	22	24	24	26	17	54	0.00%	9.0/sec	18.55	0.00

Operación Norma – Segunda Ejecución

Aggregate Report

Name: Aggregate Report (Percentiles de Latencia)

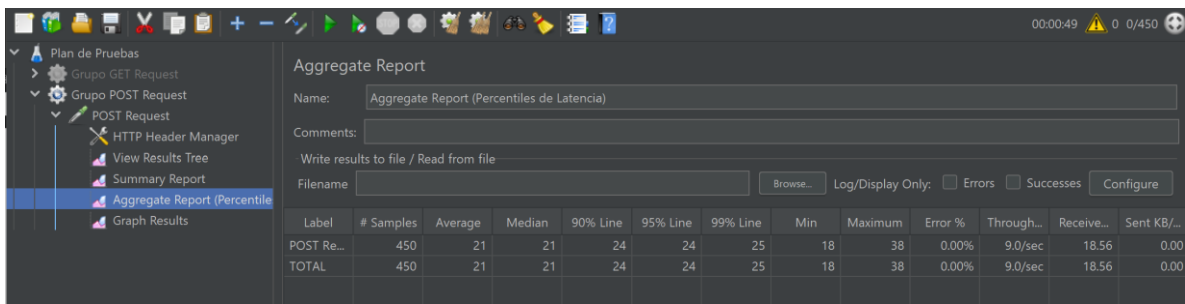
Comments:

Write results to file / Read from file

Filename: Browse... Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Through...	Receive...	Sent KB/...
POST Re...	450	21	21	23	24	25	17	39	0.00%	9.0/sec	18.55	0.00
TOTAL	450	21	21	23	24	25	17	39	0.00%	9.0/sec	18.55	0.00

Operación Norma – Tercera Ejecución



Aggregate Report

Name: Aggregate Report (Percentiles de Latencia)

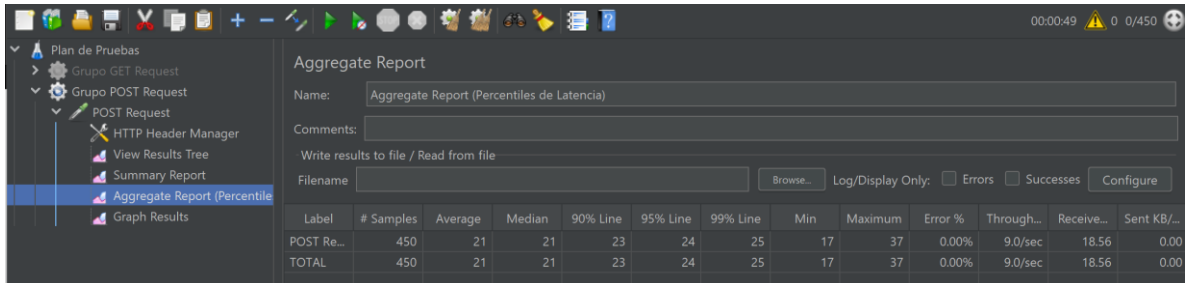
Comments:

Write results to file / Read from file

Filename: Browse... Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Through...	Receive...	Sent KB/...
POST Re...	450	21	21	24	24	25	18	38	0.00%	9.0/sec	18.56	0.00
TOTAL	450	21	21	24	24	25	18	38	0.00%	9.0/sec	18.56	0.00

Operación Norma – Cuarta Ejecución



Aggregate Report

Name: Aggregate Report (Percentiles de Latencia)

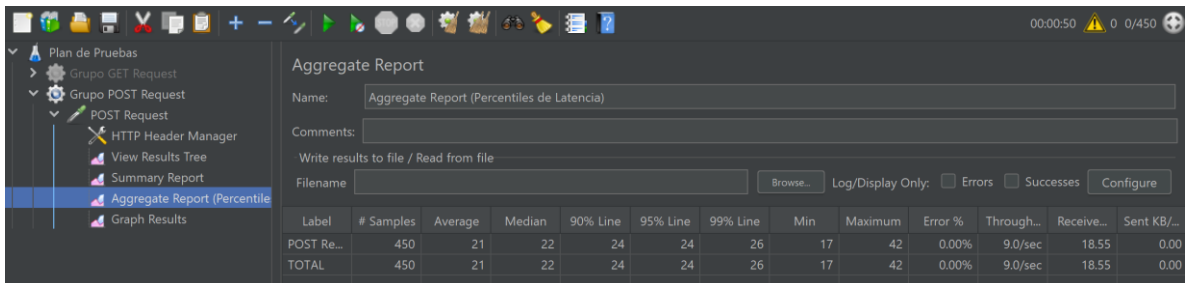
Comments:

Write results to file / Read from file

Filename: Browse... Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Through...	Receive...	Sent KB/...
POST Re...	450	21	21	23	24	25	17	37	0.00%	9.0/sec	18.56	0.00
TOTAL	450	21	21	23	24	25	17	37	0.00%	9.0/sec	18.56	0.00

Operación Norma – Quinta Ejecución



Aggregate Report

Name: Aggregate Report (Percentiles de Latencia)

Comments:

Write results to file / Read from file

Filename: Browse... Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Through...	Receive...	Sent KB/...
POST Re...	450	21	22	24	24	26	17	42	0.00%	9.0/sec	18.55	0.00
TOTAL	450	21	22	24	24	26	17	42	0.00%	9.0/sec	18.55	0.00

Alta Carga – Primera Ejecución

```
st-api-load_tests> python scripts/load_test.py --endpoint POST --users 1500 --ramp-up 75 --duration 60
=====
Resumen de la prueba – POST
=====
Total requests:      11554
Duracion real (s):   153.8
Throughput (req/s):  75.13
Latencia promedio:   11269.29 ms
Latencia p95:        37321.44 ms
Latencia p99:        58616.87 ms
Error %:              0.02%
Resultados guardados en: results_post.csv
=====
```

Alta Carga – Segunda Ejecución

```

st-api-load_tests> python scripts/load_test.py --endpoint POST --users 1500 --ramp-up 75 --duration 60
=====
Resumen de la prueba – POST
=====
Total requests:      11392
Duracion real (s):   163.3
Throughput (req/s):  69.77
Latencia promedio:   10016.39 ms
Latencia p95:        21669.76 ms
Latencia p99:        24395.33 ms
Error %:             0.14%
Resultados guardados en: results_post.csv
=====

```

Alta Carga – Tercera Ejecución

```

st-api-load_tests> python scripts/load_test.py --endpoint POST --users 1500 --ramp-up 75 --duration 60
=====
Resumen de la prueba – POST
=====
Total requests:      13391
Duracion real (s):   149.9
Throughput (req/s):  89.34
Latencia promedio:   9626.67 ms
Latencia p95:        32295.29 ms
Latencia p99:        50234.90 ms
Error %:             0.01%
Resultados guardados en: results_post.csv
=====

```

Muy Alta Carga – Primera Ejecución

```

st-api-load_tests> python scripts/load_test.py --endpoint POST --users 3000 --ramp-up 100 --duration 60
=====
Resumen de la prueba – POST
=====
Total requests:      14293
Duracion real (s):   213.5
Throughput (req/s):  66.94
Latencia promedio:   14459.29 ms
Latencia p95:        48153.47 ms
Latencia p99:        75590.30 ms
Error %:             0.01%
Resultados guardados en: results_post.csv
=====

```

Muy Alta Carga – Segundo Ejecución

```

st-api-load_tests> python scripts/load_test.py --endpoint POST --users 3000 --ramp-up 100 --duration 60
=====
Resumen de la prueba – POST
=====
Total requests:      13277
Duracion real (s):   217.7
Throughput (req/s):  61.00
Latencia promedio:   14006.47 ms
Latencia p95:        46160.39 ms
Latencia p99:        68169.31 ms
Error %:             0.00%
Resultados guardados en: results_post.csv
=====

```

Muy Alta Carga – Tercera Ejecución

```

st-api-load_tests> python scripts/load_test.py --endpoint POST --users 3000 --ramp-up 100 --duration 60
=====
Resumen de la prueba – POST
=====
Total requests:      12910
Duracion real (s):   238.4
Throughput (req/s):  54.15
Latencia promedio:   14480.16 ms
Latencia p95:        28251.89 ms
Latencia p99:        40828.77 ms
Error %:              0.05%
Resultados guardados en: results_post.csv
=====

```

Estrés – Primera Ejecución

```

st-api-load_tests> python scripts/load_test.py --endpoint POST --users 7500 --ramp-up 150 --duration 60
=====
Resumen de la prueba – POST
=====
Total requests:      18405
Duracion real (s):   391.1
Throughput (req/s):  47.06
Latencia promedio:   17011.11 ms
Latencia p95:        30273.41 ms
Latencia p99:        34569.11 ms
Error %:              38.84%
Resultados guardados en: results_post.csv
=====

```

Estrés – Segunda Ejecución

```

st-api-load_tests> python scripts/load_test.py --endpoint POST --users 7500 --ramp-up 150 --duration 60
=====
Resumen de la prueba – POST
=====
Total requests:      16797
Duracion real (s):   383.1
Throughput (req/s):  43.84
Latencia promedio:   22378.89 ms
Latencia p95:        73381.39 ms
Latencia p99:        110505.33 ms
Error %:              0.01%
Resultados guardados en: results_post.csv
=====

```

Estrés – Tercera Ejecución

```

st-api-load_tests> python scripts/load_test.py --endpoint POST --users 7500 --ramp-up 150 --duration 60
=====
Resumen de la prueba – POST
=====
Total requests:      15817
Duracion real (s):   386.8
Throughput (req/s):  40.90
Latencia promedio:   23242.76 ms
Latencia p95:        75555.56 ms
Latencia p99:        114520.40 ms
Error %:              0.01%
Resultados guardados en: results_post.csv
=====

```

Estrés Fuerte – Primera Ejecución

```

st-api-load_tests> python scripts/load_test.py --endpoint POST --users 18000 --ramp-up 200 --duration 60
=====
Resumen de la prueba – POST
=====
Total requests:      22824
Duracion real (s):   282.3
Throughput (req/s):  80.84
Latencia promedio:   22345.68 ms
Latencia p95:        37608.89 ms
Latencia p99:        67846.80 ms
Error %:             52.32%
Resultados guardados en: results_post.csv
=====

```

Estrés Fuerte – Segunda Ejecución

```

st-api-load_tests> python scripts/load_test.py --endpoint POST --users 18000 --ramp-up 200 --duration 60
=====
Resumen de la prueba – POST
=====
Total requests:      23924
Duracion real (s):   285.6
Throughput (req/s):  83.77
Latencia promedio:   22034.73 ms
Latencia p95:        36618.69 ms
Latencia p99:        76624.11 ms
Error %:             50.71%
Resultados guardados en: results_post.csv
=====

```

Estrés Fuerte – Tercera Ejecución

```

st-api-load_tests> python scripts/load_test.py --endpoint POST --users 18000 --ramp-up 200 --duration 60
=====
Resumen de la prueba – POST
=====
Total requests:      25238
Duracion real (s):   277.3
Throughput (req/s):  91.00
Latencia promedio:   18385.41 ms
Latencia p95:        32146.26 ms
Latencia p99:        48198.73 ms
Error %:             55.11%
Resultados guardados en: results_post.csv
=====

```

Estrés Fuerte – Cuarta Ejecución

```

st-api-load_tests> python scripts/load_test.py --endpoint POST --users 18000 --ramp-up 200 --duration 60
=====
Resumen de la prueba – POST
=====
Total requests:      25073
Duracion real (s):   278.1
Throughput (req/s):  90.16
Latencia promedio:   19789.92 ms
Latencia p95:        31811.87 ms
Latencia p99:        44118.05 ms
Error %:             49.86%
Resultados guardados en: results_post.csv
=====

```

Estrés Fuerte – Quinta Ejecución

```

st-api-load_tests> python scripts/load_test.py --endpoint POST --users 18000 --ramp-up 200 --duration 60
=====
Resumen de la prueba – POST
=====
Total requests:      28440
Duracion real (s):   277.7
Throughput (req/s):  102.41
Latencia promedio:   19532.90 ms
Latencia p95:        28547.79 ms
Latencia p99:        51184.50 ms
Error %:              69.59%
Resultados guardados en: results_post.csv
=====

```

Análisis de resultados

1. Punto de inflexión

El ASR que primero se incumple es el ASR 1 (GET, p99 < 1000 ms). La latencia p99 se mantiene saludable durante Smoke, Baja carga, Carga media y Operación normal (21 ms en promedio con 450 threads, muy por debajo del límite), pero al pasar a Alta carga (1500 threads) se dispara a un promedio de 49.185 ms p99 — casi 50 veces el límite del ASR. La degradación no se detiene ahí: sigue subiendo a 66.072 ms en Muy alta carga (3000), 103.366 ms en Estrés (7500) y 167.165 ms en Estrés fuerte (18000). El GET nunca reporta errores HTTP (0.00% en todos los escenarios), por lo que su incumplimiento es puramente de latencia, no de disponibilidad.

El ASR 2 (POST, error % ≤ 2%) se mantiene dentro del límite hasta Muy alta carga (3000 threads, error promedio 0.02%), pero se rompe de forma abrupta en Estrés (7500 threads), con un salto a 12.95% de error promedio (38.84% en su primera corrida), y empeora hasta 55.5% en Estrés fuerte (18000, el escenario que representa el evento de promociones real de 5000 req/min). En conclusión: el GET incumple su ASR con una carga mucho menor (1500 threads) que el POST (7500 threads), por lo que el GET es el primer eslabón en romperse, aunque el POST es el que produce el síntoma más visible para el usuario final (pedidos rechazados).

2. Cuello de botella y patrón de degradación

Para el GET, el código de `producto.repository.ts` resuelve toda la lógica de negocio (histórico de pedidos, promoción vigente y disponibilidad por zona) en una sola consulta SQL con tres subconsultas EXISTS correlacionadas — todo el trabajo pesado ocurre dentro de PostgreSQL, no en la capa de aplicación. Como no existen índices sobre `pedidos.tiendald`, `items_pedido.productold`, `catalogos.zona` ni `disponibilidad_zona.productold`, cada EXISTS obliga a un escaneo cada vez más costoso a medida que crece la concurrencia y el número de conexiones compitiendo

por el mismo pool de solo 10. El salto entre 450 y 1500 threads (de 21 ms a 49.185 ms) es abrupto, no gradual, lo que sugiere que se cruza un umbral de saturación del pool de conexiones más que un deterioro progresivo del plan de consulta.

Para el POST, `pedido.service.ts` y `pedido.repository.ts` ejecutan una escritura transaccional (INSERT en pedidos + un INSERT por cada ítem en `items_pedido`), es decir, múltiples round-trips a la base de datos dentro de una sola transacción por pedido. Bajo alta concurrencia, cada transacción abierta retiene una conexión del mismo pool de 10 que usa el GET, así que ambos endpoints compiten por el mismo recurso escaso. El patrón de degradación del error % también es abrupto (de 0.02% a 12.95% entre Muy alta carga y Estrés) en lugar de una curva suave, consistente con la hipótesis de que el pool de conexiones se agota de golpe una vez que la cantidad de transacciones concurrentes supera su capacidad, y no con un deterioro paulatino de consultas cada vez más lentas.

Un dato adicional a considerar: en Estrés fuerte (18.000 usuarios) el GET reporta p99 de 167 segundos sin ningún error, pese a que el script tiene un timeout de 10 segundos por request. Esto es evidencia de que, a esa escala de concurrencia, parte del cuello de botella podría estar en el propio generador de carga (agotamiento de sockets o del pool de conexiones del cliente Python) y no únicamente en el backend — lo cual respalda directamente la hipótesis de "límites del generador de carga" planteada en la Pregunta 3.

3. Distribución de datos

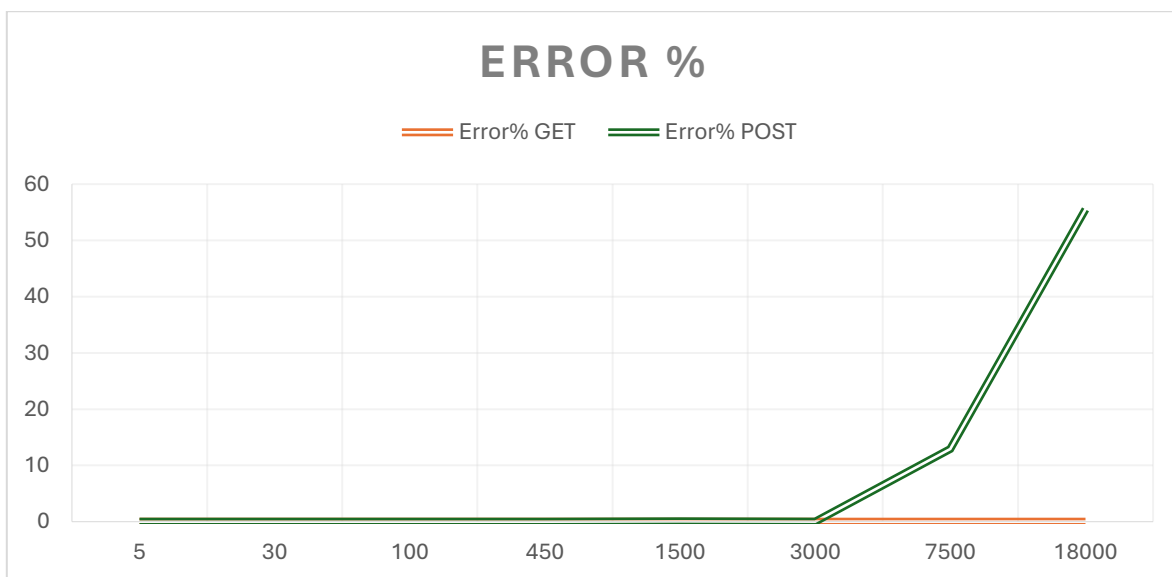
Todos los escenarios de la matriz (JMeter y script Python) se ejecutaron contra la misma configuración de `load-seed.yaml`: 200 tiendas y 10 zonas con distribución Pareto (`peso_cabeza=0.8`, `fraccion_cabeza=0.2`), 1.200 productos, 300 catálogos, 150 promociones activas y 20.000 pedidos históricos distribuidos de forma no uniforme entre tiendas. Se mantuvo esta misma distribución sesgada en toda la campaña de pruebas (no se cambió entre escenarios), en línea con la Pregunta 2: el 20% de tiendas y zonas concentra el 80% de la actividad, lo que asegura que las consultas del GET realmente golpeen filas calientes repetidas y que los resultados no sean artificialmente optimistas por un working set uniforme.

4. Arquitectura

El diseño monolítico no favorece el cumplimiento de estos dos ASR bajo carga alta: GET y POST comparten el mismo proceso Node.js, la misma instancia de PostgreSQL y, sobre todo, el mismo pool de 10 conexiones. Cuando el POST empieza a saturar ese pool en los escenarios de Estrés, le resta capacidad al GET, y viceversa — no hay aislamiento de fallos entre los dos flujos de negocio, a pesar de que tienen perfiles de

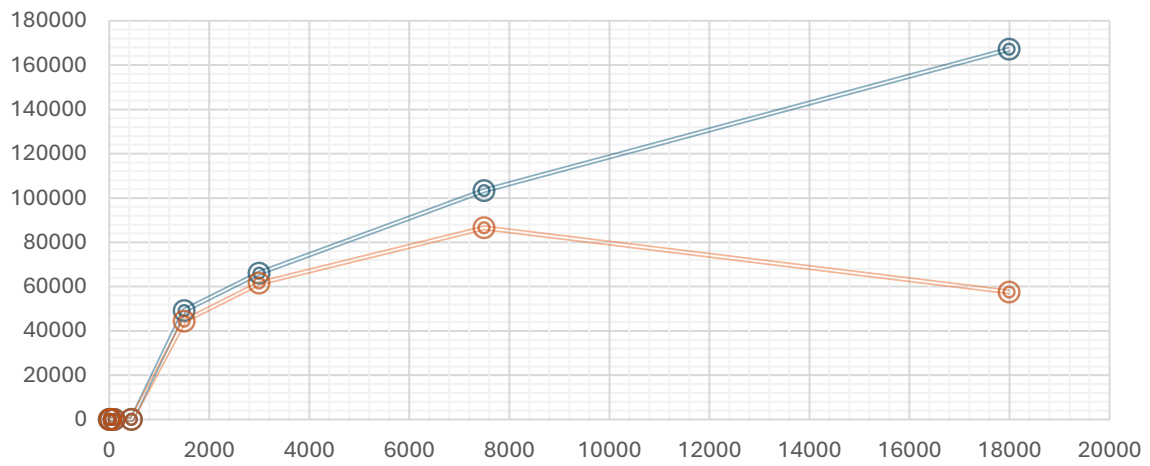
carga y criticidad distintos (lectura de catálogo vs. escritura transaccional de ingresos). Para cumplir ambos ASR de forma sostenible recomendaríamos: (1) separar el pool de conexiones por tipo de operación o incluso separar el servicio de catálogo/disponibilidad del servicio de pedidos, siguiendo un estilo más cercano a microservicios; (2) agregar los 4 índices faltantes, que es la mejora de menor costo y mayor impacto inmediato para el ASR 1; y (3) para el ASR 2, incorporar una cola de mensajes (por ejemplo un patrón de escritura asíncrona con confirmación diferida) que absorba los picos de pedidos sin depender de que cada request HTTP mantenga una transacción abierta hasta el final.

Escenario	Threads	p99 GET	p99 POST	Error% GET	Error% POST
<i>Smoke</i>	5	49	88	0.00	0.00
<i>Baja Carga</i>	30	85	86	0.00	0.00
<i>Carga media</i>	100	54	39	0.00	0.00
<i>Operación normal</i>	450	21	25.4	0.00	0.00
<i>Alta carga</i>	1500	49184.67	44415	0.00	0.057
<i>Muy Alta Carga</i>	3000	66072.33	61529	0.00	0.02
<i>Estrés</i>	7500	103365.67	86531.33	0.00	12.95
<i>Estrés Fuerte</i>	18000	167165.2	57594	0.00	55.518



P99

—○— p99 GET —○— p99 POST



Prompt utilizado para generar el script

Vamos a realizar pruebas de performance. Necesito un script en Python para ejecutar pruebas de carga contra un backend NestJS usando HTTP.

Contexto:

- Base URL: `http://localhost:3000`
- Endpoints a probar:
 - GET (lectura pesada, múltiples JOINS vía EXISTS):
`/logistics/tenderos/productos-disponibles?tiendaId=<uuid>&zona=<zona>`
 - POST (escritura transaccional): `/logistics/pedidos`
- IDs fijos disponibles en el seed de la base (`src/datasources/seed.sql`):
`tiendaId=bbbbbbbbb-bbbb-4bbb-8bbb-bbbbbbbbbbbb`, `zona="Zona Norte"`,
`monedaId=cccccccc-cccc-4ccc-8ccc-cccccccccccc`,
`productoId=aaaaaaaa-aaaa-4aaa-8aaa-aaaaaaaaaaaa` (único ID de producto garantizado por el seed fijo; el resto de productos se generan con UUID aleatorio en tiempo de seed, así que no son predecibles).

Requerimientos del script:

- * Debe permitir configurar por argumentos CLI:
 - `--endpoint` (GET o POST)
 - `--users` (conurrencia objetivo, es decir cuántos "usuarios virtuales" en paralelo)
 - `--ramp-up` (segundos para ir levantando esos `--users` de forma escalonada, no todos de golpe)
 - `--duration` (segundos que se sostiene la concurrencia máxima una vez terminado el `ramp-up`)
- * Modelo de carga: cada "usuario virtual" debe mandar peticiones EN BUCLE CONTINUO (sin tiempo de espera entre una y otra) desde que se activa (según el `ramp-up`) hasta que termine `ramp-up + duration`. Esto es para estresar el backend de forma sostenida y encontrar el punto de inflexión real, no para simular un patrón de "pensamiento humano" con pausas.
- * Para POST, arma un body de "pedido grande" con más de 20 ítems (`productoId`, `cantidad`, `precioUnitario`, `descuento`, `monedaId`), usando un identificador único (UUID) en cada request para no violar la restricción

de unicidad del campo `identificador`. Antes de arrancar la prueba, intenta obtener productId reales llamando GET /logistics/productos; si esa llamada falla o no hay suficientes productos, cae en el productId fijo del seed repetido para los 20+ ítems.

- * Debe registrar por cada request: timestamp (ISO), status_code, latency_ms, y un campo error (vacío si fue exitosa; describe si fue timeout, error de conexión, o código HTTP ≥ 400).
- * Al final debe calcular e imprimir en consola:
 - total de requests enviadas
 - throughput real (req/s), calculado sobre el tiempo total real de ejecución (no el nominal)
 - latencia promedio, p95 y p99
 - Error % (status ≥ 400 + timeouts + errores de conexión, todos cuentan como error)
- * Debe exportar los resultados a un CSV llamado results_get.csv o results_post.csv según el endpoint probado, con columnas: timestamp_iso, status_code, latency_ms, error

Restricciones:

- Usa solo la librería estándar de Python, más EXACTAMENTE una librería externa: httpx, con async/await (asyncio).
- Maneja explícitamente timeouts (httpx.TimeoutException) y errores de conexión (httpx.RequestError), regístralos como error, no dejes que tumben el script completo.
- No generes gráficas (matplotlib) – solo el CSV, las voy a graficar yo manualmente en Excel para el informe.
- Ajusta los límites de conexión del cliente httpx (httpx.Limits) para que soporte hasta 18000 usuarios concurrentes sin quedarse corto de conexiones disponibles en el pool del propio cliente.

Incluye instrucciones de ejecución con ejemplos, por ejemplo:

```
python scripts/load_test.py --endpoint GET --users 1500 --ramp-up 75 --duration 60
python scripts/load_test.py --endpoint POST --users 3000 --ramp-up 100 --duration 60
```